

Ecosistema de automatización IA autónomo

Solicitud de cuidado entrante » calificación con IA sobre la base de datos » validación humana » salida multicanal

Entregable	Dónde
Repositorio público	https://github.com/rubenvg11/cuidda-ecosistema-automatizacion-ia
Panel de control en vivo	https://rubenvg11.github.io/cuidda-ecosistema-automatizacion-ia/
Blueprints de Make	blueprints/ — los dos .json completos, sin fragmentar
Esquemas de transferencia	schemas/ — 6 archivos JSON Schema 2020-12
Evidencia del test	evidencias/ — mapa de arquitectura y capturas de las corridas

Qué resuelve

Cuidda conecta familias con personal de salud verificado a domicilio en Trujillo. Las solicitudes llegan por correo, en lenguaje natural: «a mi mamá le dan de alta pasado mañana y no podemos dejarla sola de noche». Alguien tiene que leer eso, cruzarlo con qué distritos cubrimos y qué turnos existen, calcular un monto, y contestar algo que no prometa lo que no podemos cumplir.

El riesgo de automatizarlo con IA no es que escriba feo. Es que invente: una tarifa, un distrito que no cubrimos, una disponibilidad inmediata. Del otro lado hay una familia decidiendo quién entra a su casa a cuidar a su madre. Un correo mal escrito no es un problema de marketing: es una familia que no vuelve.

Por eso el ecosistema ataca el riesgo por tres lados: le saca a la IA la posibilidad de inventar —le inyecta la cobertura y el catálogo reales desde la base de datos y le prohíbe salirse de ahí—, le saca la decisión de negocio —la ruta que toma cada solicitud la deriva el flujo de los campos extraídos, no el modelo— y le saca la posibilidad de enviar sola: ninguna rama puede escribirle a una familia sin que antes una persona haya aprobado el texto.

El stack

Capa	Herramienta	Rol
Orquestador	Make · zona us2	2 escenarios, 41 módulos en total
Base de datos y memoria	Airtable	6 tablas relacionadas: el estado, las reglas y el registro
Procesamiento IA	Claude Haiku 4.5	clasificar y redactor, con ruteo que evita la llamada cara
Canal de salida	Gmail + Slack	la familia recibe correo; el equipo aprueba en Slack

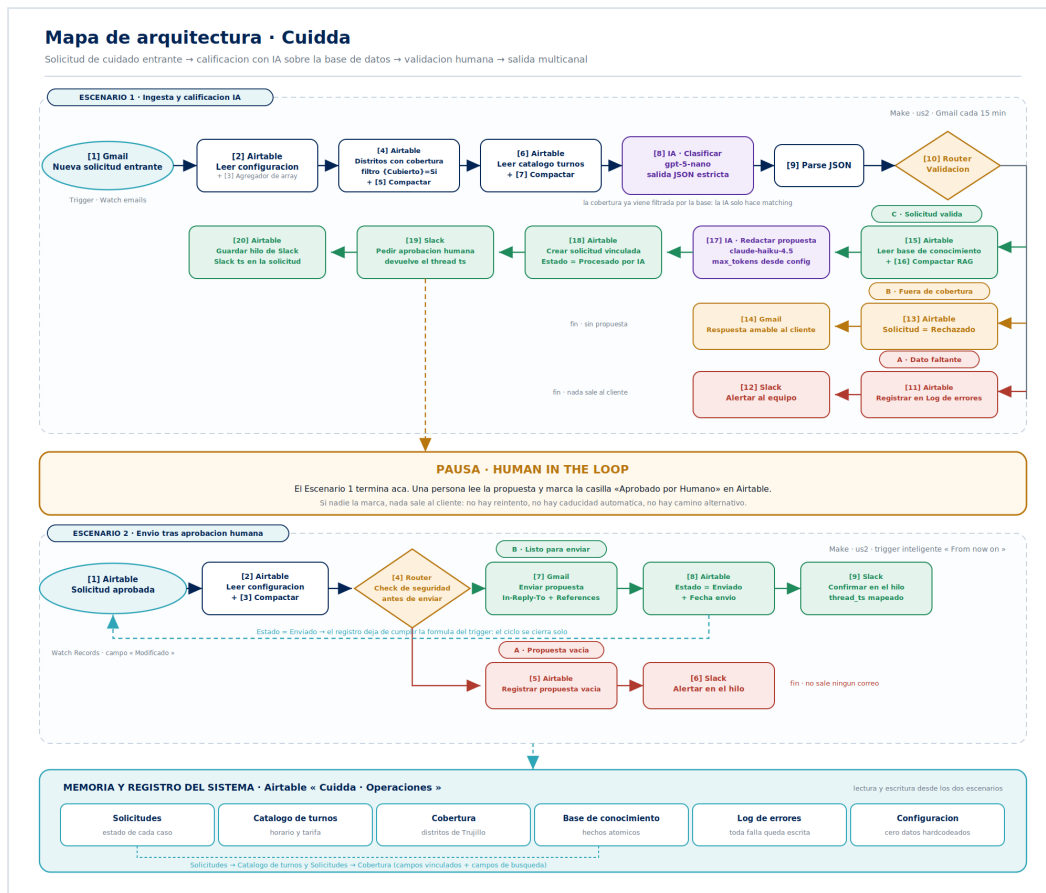
Por qué Gmail y Slack, y no WhatsApp

La consigna permitía Gmail, Slack o WhatsApp API. Se eligieron los dos primeros porque la entrada del sistema ya es un correo: responder por el mismo canal mantiene el hilo y no obliga a la familia a instalar nada. WhatsApp Business exigiría una plantilla aprobada por Meta, y el mensaje de propuesta de Cuidda es demasiado largo y variable para encajar en una plantilla. Queda documentado como el siguiente canal a sumar, no como un olvido.

1 · Mapa de arquitectura

El diagrama completo: triggers, routers, APIs, nodos de IA y destino de los datos. No está dibujado a mano — lo genera herramientas/diagrama.py, así que cambiar el flujo y volver a correr el script mantiene el mapa sincronizado.

Mapa completo del ecosistema



Escenario 1 arriba, la pausa humana en el medio, Escenario 2 abajo y la capa de memoria al pie. Cada caja lleva el mismo numeral que el módulo en Make.

Escenario 1 · Ingesta y calificación IA — 28 módulos, 3 rutas

Trigger inteligente. El módulo [1] observa Gmail con el filtro is:unread subject:(solicitud de cuidado) y arranca en From now on: Make guarda la marca de tiempo del arranque y nunca procesa correo viejo. Además marca como leído lo que procesa, así que un correo no puede entrar dos veces.

Carga de contexto. Antes de llamar a la IA, el flujo lee de Airtable la configuración [2], los distritos con cobertura [4] y el catálogo de turnos [6], y los compacta [3][5][7]. Si mañana Cuidda abre un distrito o cambia una tarifa, el prompt cambia solo.

La regla de negocio no la evalúa la IA

El módulo [4] no lee la tabla Cobertura entera: la lee con la fórmula {Cubierto} = "Si". El prompt recibe una lista que ya filtró la base de datos, y la única pregunta que le queda a la IA es «¿el lugar que escribió esta familia está en esta lista?». No es un detalle de implementación: en el test de estrés, pedirle a un modelo barato que leyera una columna Si/No por línea falló 3 de 5 veces. La regla de quién tiene cobertura vive en Airtable, donde se audita y se cambia sin tocar un prompt.

Motor de IA y ruteo. El módulo [8] devuelve un JSON de 12 claves, [9] lo parsea y el router [10] decide con una expresión que se calcula sobre esos campos:

```
{ { if(length(trim(9.distrito_mencionado)) = 0; "A";
    if(length(trim(9.turno)) = 0; "A";
    if(length(trim(9.distrito)) = 0; "B"; "C")) ) }
```

Ruta	Cuándo	Qué hace
A · Dato faltante	no dice dónde vive o no dice qué turno	[11] registra en Log de errores con la lista de lo que faltó · [12] avisa por Slack. No crea solicitud y no gasta la llamada de redacción.
B · Fuera de cobertura	el correo está completo pero el lugar no está en la lista de la base	[13] crea la solicitud como Rechazado con el motivo · [14] responde a la familia por Gmail, en el mismo hilo, nombrando el lugar tal como ella lo escribió. Sin propuesta: no tiene sentido redactar algo que no se puede cumplir.
C · Solicitud válida	completo y dentro de cobertura	[15] lee la base de conocimiento · [16] la compacta · [17] redacta la propuesta · [18] crea la solicitud vinculada · [19] pide aprobación en Slack · [20] guarda el ts de ese mensaje para responder después en el mismo hilo.

Dónde termina la máquina

El Escenario 1 termina en [19]/[20]. No hay rama que envíe una propuesta. La única salida hacia el cliente en todo el escenario es el correo de la ruta B, que es un «todavía no llegamos a tu zona» — no una propuesta comercial.

Escenario 2 · Envío tras aprobación humana — 13 módulos, 2 rutas

El trigger [1] observa la tabla Solicitudes por el campo Modificado con la fórmula AND({Aprobado por Humano} = 1, {Estado} = "Aprobado"), también en From now on. Un registro que nadie aprobó nunca entra al flujo: no es un filtro posterior, es la condición de entrada.

Ruta	Condición	Qué hace
A · Propuesta vacía	length(trim(Propuesta generada)) = 0	[5] registra el caso · [6] avisa en el hilo de Slack. No sale ningún correo.

Ruta	Condición	Qué hace
B · Listo para enviar	la propuesta tiene contenido	[7] envía por Gmail con In-Reply-To y References para caer en el hilo original · [8] cierra el ciclo con Estado = Enviado y Fecha envío · [9] confirma en el hilo de Slack.

Nombres de nodo

Todos los módulos llevan en Make el mismo nombre y el mismo numeral que en este mapa. Un error registrado en la tabla Log de errores dice, por ejemplo, [17] IA Redactar propuesta: se ubica en el diagrama sin adivinar.

2 · Estructuras de datos documentadas

Dos cosas distintas: cómo está modelada la base, y qué forma tiene el dato mientras viaja entre nodos.

El modelo en Airtable

Tabla	Rol
Solicitudes	El caso. Una fila por familia que escribió.
Cobertura	Los 9 distritos de Trujillo: si hay personal y cuánto recarga la movilidad.
Catalogo de turnos	Los 5 turnos con su horario, tarifa y notas operativas.
Base de conocimiento	10 entradas atómicas verificadas. Es el RAG.
Log de errores	Toda falla, de API o de validación, queda escrita acá.
Configuracion	12 pares clave-valor leídos en runtime. Es lo que hace que no haya datos hardcoded.

Las relaciones, y por qué existen

+-----+ Cobertura Distrito · Cubierto Recargo movilidad +-----+		+-----+ link +-----+
+-----+ Solicitudes Codigo · Estado Prioridad · Monto +-----+	<- link	+-----+ Catalogo de turnos Turno · Horario Tarifa soles +-----+

lookups que bajan a Solicitudes:
 Cubierto (from Distrito) · Recargo movilidad soles (from Distrito)
 Horario (from Turno) · Tarifa soles (from Turno)

El requisito era evitar datos aislados, y acá se cumple de forma concreta: en Solicitudes no hay una columna de texto que diga «Huanchaco» y otra que diga «20». Hay un campo vinculado al registro real de Cobertura, y el recargo llega por lookup. Si mañana Huanchaco pasa a recargo 25, las solicitudes existentes lo reflejan sin tocar nada. Si Huanchaco deja de tener cobertura, el dato viaja solo — y el módulo [4] deja de inyectarlo en el prompt en la corrida siguiente.

Lo mismo con los turnos: la tarifa de «Noche» vive en un único lugar. El campo Monto estimado de una solicitud se audita contra Tarifa soles (from Turno) más Recargo movilidad soles (from Distrito) sin salir de la fila. En el test de estrés esa verificación se hizo fila por fila y cerró exacta.

Ciclo de vida de una solicitud

```

Pendiente --> Procesado por IA --> Aprobado --> Enviado
          |               ^
          |               +--- una persona, a mano
          |               (fuera de cobertura)
          +---> Rechazado
          +---> Error      (falla registrada)

```

Enviado es un estado absorbente: es lo que corta el loop del Escenario 2.

Los esquemas JSON de transferencia

Cada salto entre nodos tiene un contrato explícito, publicado en schemas/ como JSON Schema 2020-12 válido y ejecutable contra un validador.

Archivo	Qué contrato describe
01 · gmail-a-clasificador	Correo entrante » módulo [8]. Incluye la lista de campos descartados por minimización de datos.
02 · clasificacion-ia	El contrato central: las 12 claves que debe devolver el clasificador, la tabla de ruteo y tres ejemplos.
03 · solicitud-airtable	El registro escrito en Solicitudes, indexado por fieldId y no por nombre.
04 · log-de-errores	El registro de error, con la directiva que corresponde a cada tipo.
05 · airtable-a-escenario-2	El handoff entre escenarios. Airtable no es solo destino: es el bus de mensajes.
06 · canales-de-salida	Payloads de Slack y Gmail, con el mapeo de Thread ID en cada uno.

El contrato que más importa

El módulo [8] devuelve texto. El [9] lo parsea. El router [10] decide. Si ese texto no es JSON válido con estas 12 claves exactas, todo lo que sigue se cae. Por eso el prompt lo declara clave por clave y el esquema lo fija con `additionalProperties: false` y enums cerrados.

```

{
  "familia": "Marco Antonio Delgado Chavez",
  "email": "marco.delgado@example.com",
  "telefono": "946307128",
  "distrito_mencionado": "el centro de Trujillo",
  "distrito": "Trujillo Centro",
  "turno": "Manana",
  "prioridad": "Media",
  "resumen": "Senora mayor que vive sola y se marea al caminar,
             necesita acompanamiento de lunes a viernes.",
  "monto_estimado": 90,
  "datos_completos": "si",
  "distrito_cubierto": "si",
  "faltantes": ""
}

```

Por qué dos claves de distrito

Hay dos claves de distrito y no una, y ahí está el corazón del ruteo. distrito_mencionado es lo que escribió la familia, sea lo que sea: «Laredo», «el centro de Trujillo», «acá por Moche». distrito solo tiene valor si ese lugar aparece en la lista de distritos con cobertura que la base inyectó en el prompt.

Eso es lo que permite que un correo completo de un distrito sin cobertura caiga en la ruta B (se le responde) y no en la ruta A (dato faltante): al router le falta un distrito válido, pero no le falta información de la familia.

Teléfono en E.164

telefono sale sin prefijo y solo con dígitos a propósito. El prefijo lo pone Make leyendo Configuracion.prefijo_telefono_pais, y el resultado se escribe como +51946307128 en un campo de tipo «Número de teléfono». Así el signo + no se pierde y el formato E.164 no depende de que la IA se acuerde de ponerlo.

Escritura por ID de campo

Make escribe usando el fieldId de Airtable, nunca el nombre visible:

```
"record": {
  "fldswL3l4BgSrPlPr": "SQL-{{formatDate(now; \"YYYYMMDD-HHmss\")}}",
  "fld80QzLKX51uSjDw": "{{9.familia}}",
  "fldtI5sKssoSkCH7G": ["{{9.distrito}}"]
}
```

Renombrar «Familia» a «Nombre de la familia» en Airtable no rompe nada. Es la diferencia entre un flujo que sobrevive a que alguien ordene la base y uno que se cae el martes siguiente.

3 · Optimización de costos

No se trata de limitar tokens. Se trata de decidir, tarea por tarea, qué modelo hace falta y qué trabajo directamente no hay que pagar — y de medir esa decisión en vez de suponerla.

Las dos tareas de IA no son la misma tarea

	[8] Extraer y clasificar	[17] Redactar la propuesta
Qué hace	Lee un correo y devuelve un JSON de 12 claves	Escribe el correo que una familia preocupada va a leer
Quién lo consume	Un router (una máquina)	Una persona
Tolerancia al error	Cero de otro tipo: si erra el distrito, la familia entra a la ruta equivocada	Cero: el tono es el producto
Se ejecuta	En el 100 % de las solicitudes	Solo en las válidas y dentro de cobertura

El experimento que cambió la arquitectura

La hipótesis inicial era la del manual: modelo barato para clasificar, modelo bueno para redactar. El clasificador arrancó en el modelo small de Make AI Toolkit —la fila más barata de la matriz— y el test de estrés lo desarmó.

Caso del test	Qué hizo el modelo barato	Consecuencia
«el centro de Trujillo»	Devolvió Trujillo, que no existe en Cobertura	Airtable creó un distrito fantasma por typecast
Huanchaco (Cubierto = Si)	Lo marcó fuera de cobertura	Se rechazó a una familia que sí se puede atender
Moche + Día completo	Monto estimado erróneo	La propuesta habría salido con un precio mal calculado

3 de 5 corridas mal. Con Claude Haiku 4.5 en el mismo prompt, las mismas cinco corridas dieron distrito exacto, cobertura correcta y montos que cierran contra la base (Trujillo Centro S/ 90, Moche S/ 175). Dos decisiones salieron de ahí:

- 1 Subir el clasificador a Haiku 4.5. Cuesta 16 veces más por llamada y sigue siendo un costo de centavos frente al de rechazar mal a una familia.
- 2 Sacarle la regla de negocio al modelo. El módulo [4] filtra la cobertura en Airtable y el router deriva la ruta de los campos extraídos. Aunque el modelo se equivoque, no puede inventar una cobertura que la base no dice.

Lo que vale de este criterio

La fila más barata de la tabla está descartada con datos, no por intuición. Esa es la diferencia entre una matriz de decisión y una tabla de precios copiada: la matriz se completa corriendo el flujo.

Medición real de tokens

Los prompts están en el blueprint y se pueden medir. La parte fija se contó sobre el texto del propio .json; la dinámica, sobre los datos reales de las tablas.

Módulo	Prompt fijo	Datos inyectados	Input total	Output
[8] Clasificar	612 tok	Cobertura 96 + Catálogo 90 + correo ~250	≈ 1 050 tok	≈ 170 tok
[17] Redactar	379 tok	Base de conocimiento 774 + solicitud 40	≈ 1 190 tok	≈ 300 tok

Matriz de decisión

Precios públicos por millón de tokens, septiembre 2026:

Modelo	Input	Output	Clasificar (1050/170)	Redactar (1190/300)
gpt-5-nano / small	\$0.05	\$0.40	\$0.000121	\$0.000180
Claude Haiku 4.5	\$1.00	\$5.00	\$0.001900	\$0.002690

Modelo	Input	Output	Clasificar (1050/170)	Redactar (1190/300)
Claude Sonnet 5	\$2.00	\$10.00	\$0.003800	\$0.005380
Claude Opus 5	\$5.00	\$25.00	\$0.009500	\$0.013450

Proyección sobre 1 000 solicitudes entrantes

Con la distribución del test de estrés: 60 % válidas, 20 % fuera de cobertura, 20 % con datos faltantes. Solo las válidas llegan al redactor.

Escenario	Clasificar	Redactar	Total / 1 000	vs. elegido
Arquitectura elegida Haiku + ruteo	\$1.90	\$1.61	\$3.51	—
Haiku sin ruteo (redactar siempre)	\$1.90	\$2.69	\$4.59	+31 %
nano para clasificar	\$0.12	\$1.61	\$1.73	−51 % y 3 de 5 mal
Todo Sonnet 5, sin ruteo	\$3.80	\$5.38	\$9.18	+162 %
Todo Opus 5, sin ruteo	\$9.50	\$13.45	\$22.95	+554 %

De dónde sale el ahorro

Con el clasificador ya fijado en Haiku, el ahorro viene de no redactar lo que no se va a enviar: las rutas A y B cortan antes del módulo [17]. Son \$1.08 cada 1 000 solicitudes, un 24 %, y crece con el porcentaje de correos incompletos o fuera de zona — justo los que más abundan cuando un negocio empieza a crecer. Contra la alternativa perezosa de un modelo grande sin ruteo, el ahorro es del 62 % (Sonnet 5) y del 85 % (Opus 5).

Por qué no se toma el ahorro más grande

Bajar el clasificador a nano ahorraría \$1.78 más cada 1 000 solicitudes. En el test equivocó el distrito, la cobertura y el monto en 3 de 5 casos. Una familia mal rechazada cuesta más que \$1.78, y un monto mal calculado en un correo firmado por Cuidda cuesta bastante más que eso.

Límite de tokens de salida

El módulo ai-tools:Ask de Make AI Toolkit no expone un parámetro de max tokens — sus únicos parámetros son la conexión y el modelo. Decirlo así es más útil que fingir un campo que no existe. El tope se aplica donde sí se puede:

Dónde	Cómo
En el prompt, con un valor leído de la base	La regla 6 del redactor dice: «Tope duro de salida: {{ get(map(3.array; "Valor"; "Clave"; "max_tokens_propuesta"); 1) }} tokens, que son unas 200 palabras. Si te estás pasando, corta, no resumas a la mitad.» El valor vive en Configuración = 600.
En el clasificador, por diseño	Devuelve 12 claves y el campo resumen tiene tope de 30 palabras. La salida no puede crecer.

Dónde	Cómo
En el contexto de entrada	Los tres Search Records tienen maxRecords: 20. Las tablas no pueden inflar el prompt sin límite aunque crezcan.

El identificador del modelo: lo que quisimos hacer y lo que se puede

La intención era leerlo de Configuracion igual que todo lo demás:

```
{{ get(map(3.array; "Valor"; "Clave"; "modelo_id_clasificacion"); 1) }}
```

No funciona, y vale la pena contarlo. El campo Model de ai-tools:Ask no es un campo mapeable del bundle: es un parámetro de configuración del módulo. Make no evalúa IML ahí en tiempo de ejecución, así que el escenario intentaba invocar un modelo llamado literalmente {{ get(map(3.array; ... y el módulo fallaba. Lo detectamos en la tabla Log de errores, en el campo Detalle, que guarda el mensaje textual del proveedor — exactamente para lo que se diseñó esa tabla.

Así que el identificador va literal en los dos módulos, y lo que sí es dinámico es el nombre legible, que se lee de Configuracion y se escribe en Solicitudes.Modelo usado en cada solicitud: el costo real sigue siendo auditable fila por fila desde el dashboard. Preferimos dejarlo escrito antes que mostrar una captura del blueprint con la expresión adentro y no aclarar que no se evalúa.

El costo de no hardcodear nada

Hay que decirlo: leer Configuracion, Cobertura y Catalogo de turnos en cada corrida cuesta 6 de las operaciones del Escenario 1. Es el precio de que ningún valor esté escrito dentro del flujo, y se paga a conciencia. El siguiente paso natural, cuando el volumen lo justifique, es cachear esas tres tablas en un Data Store de Make refrescado una vez al día: mismas garantías, seis operaciones menos por solicitud. No está implementado hoy porque a este volumen la diferencia es de centavos y la lectura directa es más fácil de auditar.

4 · Seguridad y resiliencia

Tres preguntas: qué datos toca el sistema, qué pasa cuando algo se rompe, y quién autoriza lo que sale hacia afuera.

Minimización de datos

Cuidda maneja datos de salud de personas mayores. El principio: cada capa recibe lo mínimo que necesita para hacer su trabajo, y nada más.

Capa	Qué recibe	Qué se descarta
Gmail » clasificador	subject, fromName, fromEmail, fullTextBody, threadId, message-id	htmlBody, cc, bcc, to, labelIds, snippet, adjuntos, sizeEstimate, historyId
Clasificador » Airtable	Las 12 claves del contrato JSON	El correo nunca se reenvía crudo a la IA de redacción
Redactor (RAG)	Distrito, turno, monto, resumen y la base de conocimiento	No recibe el correo original, ni el email, ni el teléfono
Slack	Nombre, prioridad, distrito, turno, monto y la propuesta	No recibe el email ni el teléfono de la familia
Dashboard público	Código, fecha, distrito, turno, prioridad, monto, modelo, estado	9 campos ocultos, todos los que contienen datos personales

Dos decisiones que vale subrayar

El redactor no ve datos de contacto. Le llega distrito, turno, monto y resumen. No necesita el correo ni el teléfono para escribir la propuesta, así que no los recibe. Si mañana el proveedor de IA cambia, lo que nunca salió de Airtable sigue sin salir.

El panel público no tiene ni un dato personal. La vista compartida de Log de errores oculta el campo Detalle justamente porque ahí sí puede aparecer un correo de remitente.

Ninguna API key vive dentro del flujo. Las cuatro conexiones son credenciales OAuth gestionadas por Make y referenciadas por id numérico. El blueprint publicado en el repo contiene ids de conexión, no secretos.

Rutas de error

Hay seis handlers repartidos en los dos escenarios, y ninguno hace lo mismo que el otro, porque los fallos no son iguales.

[8] IA · Extraer y clasificar » Break con reintento

```
onerror -> [8E] Airtable · Registrar fallo de la IA    (Tipo = "Fallo de API de IA")
         -> [8E] Slack   · Alertar fallo de la IA    (con el error textual)
         -> [8E] Break   · Detener con reintento    (3 intentos, cada 15 s)
```

Si el clasificador no responde no hay nada que hacer aguas abajo: sin JSON no hay ruta. Break congela la ejecución en Incomplete executions y Make la reintenta sola. No se escribe ninguna solicitud a medias.

[9] Parsear clasificación » Break con reintento

Este handler no estaba en el diseño original: lo pidió el test. Claude devuelve el JSON envuelto en un bloque de código aunque el prompt lo prohíba, y el parser respondía «Source is not valid JSON. Code: DataError». Se arregló por los dos lados: el flujo normaliza la respuesta antes de parsear —misma idea que el trim() del Escenario 2: normalizar y después comparar— y, si aun así falla, hay una ruta de error que lo registra con la respuesta cruda del modelo, avisa por Slack y deja la ejecución para reintentar.

```
{{ trim(replace(replace(8.answer; "`json"; ""); "`"; "")) }}
```

[17] IA · Redactar propuesta » Resume

```
onerror -> [17E] Airtable · Registrar fallo de redaccion
        -> [17E] Resume · Continuar con aviso
            answer = "PROPUESTA NO GENERADA - la API de IA fallo.
                    Revisar a mano antes de aprobar."
```

Acá la decisión es la contraria, y es deliberada. La solicitud ya fue validada: perderla porque el redactor se cayó sería el peor resultado posible. Resume inyecta un texto de aviso y el flujo sigue: la solicitud se crea igual y el mensaje de Slack dice en el cuerpo que la propuesta no se generó. Preferimos una propuesta vacía y visible antes que una solicitud perdida en silencio.

Handler	Directiva	Por qué esa y no otra
[8] fallo de IA	Break 3×15 s	Sin JSON no hay ruta posible. Reintentar sí arregla el problema.
[9] JSON inválido	Break 3×15 s	El modelo puede acertar en el segundo intento. Se guarda la respuesta cruda para poder depurarlo.
[11] dato faltante	sin reintento	No es un fallo de sistema, es un correo incompleto. Reintentar no lo arregla.
[17] fallo de redacción	Resume	La solicitud ya vale. Perderla es peor que entregarla sin texto.
[7] fallo de Gmail	Break 3×15 s	El estado no avanza, así que el registro vuelve a entrar solo.
[9] fallo de Slack	Resume	El correo ya salió. Que Slack no conteste es molesto, no grave.

Los dos escenarios tienen Store incomplete executions = Yes, que es el requisito técnico para que la directiva Break con reintento funcione.

Lo importante del handler de Gmail es lo que no pasa: como Break detiene el flujo, el módulo [8] del Escenario 2 nunca corre, así que el Estado se queda en Aprobado. El registro sigue cumpliendo la fórmula del trigger y vuelve a entrar cuando Gmail se recupere. Nunca se marca como «Enviado» un correo que no salió.

Punto de validación humana

El sistema no puede escribirle una propuesta a una familia por su cuenta. La frontera es física, no una convención:

- 1 El Escenario 1 escribe la solicitud con Estado = «Procesado por IA» y Aprobado por Humano = false. Siempre. No hay rama que lo ponga en true.

- 2 Publica en Slack la propuesta completa con el código de la solicitud y la frase «Nada sale al cliente hasta que alguien marque Aprobado por Humano y ponga el Estado en Aprobado en Airtable».
- 3 El Escenario 2 solo existe para registros que cumplen la fórmula. Los no aprobados ni siquiera entran al flujo.

Doble llave

Hacen falta dos acciones humanas distintas —marcar la casilla y cambiar el estado— para que un correo salga. Un clic accidental en la casilla no dispara nada.

Check de seguridad

Filtro para evitar loops infinitos

El Escenario 2 termina poniendo Estado = Enviado. Eso toca el campo Modificado, que es el trigger field, así que el registro vuelve a pasar por el trigger —pero ya no cumple la fórmula, porque Estado dejó de ser «Aprobado». El ciclo se corta solo. No depende de un contador, ni de un delay, ni de una lista de procesados: la condición de entrada deja de ser verdadera por efecto de la propia acción.

Comparación de tipos correcta en los filtros

Las tres rutas del Escenario 1 comparan texto contra texto con text:equal sobre un enum cerrado de tres valores, nunca con notequal sobre un campo libre. Esto corrige directamente la observación sobre el filtro de la ruta «Baja», donde un notequal se solapaba con la ruta «Alta». Y el valor comparado no lo escribe el modelo: lo deriva el flujo.

Ruta	Condición
A · Dato faltante	ruta = "A"
B · Fuera de cobertura	ruta = "B"
C · Solicitud válida	ruta = "C"

En el router del Escenario 2 el problema era distinto: había que preguntar si un campo de texto largo está vacío. En vez de comparar el campo contra una cadena vacía —que es donde se cuelan los espacios y los saltos de línea— se normaliza primero y se compara contra un enum:

```
{{if(length(trim(1.`Propuesta generada`)) = 0; "vacía"; "lista")}}
```

Ruta A pregunta = "vacía", ruta B pregunta = "lista". Texto contra texto, dos valores posibles, sin ambigüedad.

El AND del blueprint se escribe de una sola forma, y no es la intuitiva

Esto costó una tarde de test y merece quedar escrito. En el JSON de un blueprint de Make, el campo conditions es un array de arrays: el array externo es un OR y el interno es un AND.

```
"conditions": [[c1], [c2]] // c1 O c2 <- parece un AND y no lo es
"conditions": [[c1, c2]] // c1 Y c2 <- el AND de verdad
```

Escrito de la primera forma, la ruta B pasaba a ser «datos_completos = si o distrito_cubierto = no», que es verdadera casi siempre: todas las solicitudes se registraban además como rechazadas y el test creaba dos filas por corrida. El síntoma en History era claro una vez que se sabía qué mirar —17 operaciones donde la ruta C sola son 15— pero el blueprint se veía perfectamente razonable.

La forma de verificarlo no es leer el JSON: es abrir el filtro en la UI de Make después de importar. Si entre las dos condiciones dice or en vez de and, el blueprint está mal aunque el escenario corra sin errores. Es la clase de bug que no rompe nada: solo hace lo incorrecto, en silencio.

Prompt dinámico con variables del sistema

Ningún prompt tiene datos fijos. La ciudad, la moneda, la firma, las horas de vigencia y el tope de tokens salen de Configuración; la cobertura, el catálogo de turnos y la base de conocimiento salen de sus tablas y se inyectan agregados. Si mañana Cuidda abre en Chiclayo, se agrega la fila y el prompt cambia solo.

Lo que todavía no está resuelto

Un informe que solo dice lo que funciona no sirve para operar. Cuatro cosas quedan abiertas:

El plan Free de Make puede sustituir el modelo	El propio módulo avisa: «Some models are available only on Make paid plans. If you're on the free plan or a trial, your scenario will run using a free model.» La elección de Claude Haiku 4.5 se respeta recién en plan pago. El campo Modelo usado registra lo que el escenario pidió, no necesariamente lo que Make ejecutó.
El monto lo sigue calculando la IA	Tarifa y recargo ya están en Airtable como lookups. Lo correcto es que Monto estimado sea un campo fórmula y que la IA no haga aritmética. Es el próximo cambio; se dejó fuera de esta entrega para no tocar el esquema en medio del test.
No hay política de retención	Los correos originales quedan en Mensaje original indefinidamente. Para un sistema que maneja datos de salud, lo correcto es vaciar ese campo a los 90 días de Fecha envío.
El motivo de rechazo lo escribe la máquina	Cuando una persona decide no aprobar una propuesta, hoy no hay campo donde deje por qué. Es el primer agujero que tataría antes de poner esto en producción.

5 · Dashboard de control

Es más que el link a la base. El panel es una página propia, con la identidad de Cuidda, que explica qué significa cada número y embebe las vistas de Airtable en vivo. No hay copias ni cachés: lo que se ve es la misma fila que el flujo acaba de escribir.

Panel público

<https://rubenvg11.github.io/cuidda-ecosistema-automatizacion-ia/>

El panel de control

Evidencia pendiente: evidencias/dashboard-recorte.png.

Los tres paneles

Panel	Qué muestra
1 · Pipeline de solicitudes	Todas las solicitudes agrupadas por Estado, con distrito, turno, prioridad, monto y modelo usado. El encabezado de cada grupo es el contador de ese estado.
2 · Cola de aprobación humana	Filtrada por Estado = «Procesado por IA»: lo que espera una decisión de una persona.
3 · Errores por tipo	Log de errores agrupado por Tipo de error, con el nodo exacto que falló.

Los cuatro indicadores

Indicador	Cómo se calcula	Dónde se lee
Solicitudes por estado	count(Solicitudes) agrupado por Estado	Panel 1 · encabezado de grupo
Cola de aprobación humana	count(Solicitudes) donde Estado = "Procesado por IA"	Panel 2 · contador inferior
Monto en juego	sum(Monto estimado)	Panel 1 · barra de resumen
Tasa de errores	$\text{count(Log de errores)} \div (\text{count(Solicitudes)} + \text{count(Log · "Dato faltante")}) \times 100$	Panel 3 ÷ Panel 1

Por qué el denominador de la tasa de errores es raro

Una solicitud con datos faltantes nunca llega a crear una fila en Solicitudes: muere en la ruta A y solo deja rastro en Log de errores. Si se dividiera por las solicitudes creadas, esos casos desaparecerían del denominador y la tasa quedaría artificialmente baja, justo en el escenario donde más importa que no lo esté. Sumarlos es lo que hace que el número signifique lo que uno cree que significa: de cada 100 correos que entraron, cuántos terminaron mal.

Minimización de datos en el panel

El panel es público, así que las vistas compartidas ocultan 9 campos de Solicitudes: Familia, Email, Telefono, Mensaje original, Resumen IA, Propuesta generada, Hilo Gmail, Message ID Gmail y Slack ts. En Log de errores se oculta Detalle. Queda visible exactamente lo que un panel de control necesita: el código, la fecha, el distrito, el turno, la prioridad, el monto, el modelo y el estado. Ni un dato personal.

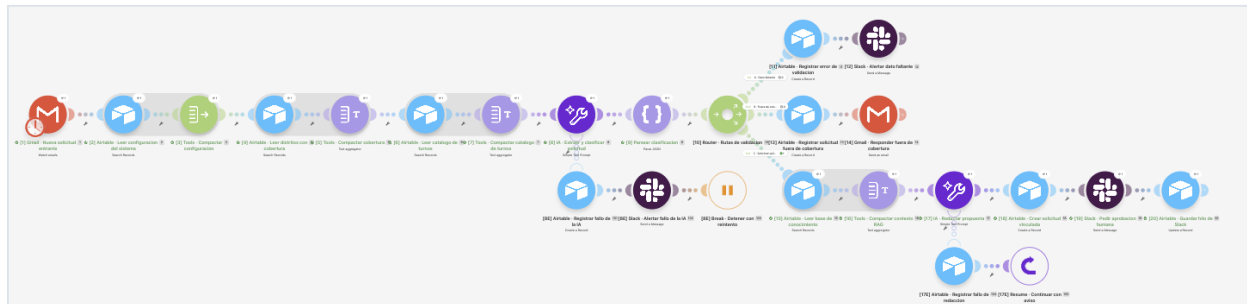
Test de estrés

Cinco ejecuciones que recorren las tres rutas del Escenario 1, incluido el camino infeliz con datos incompletos. Todas se corrieron sobre la base limpia, una por correo, con Run once, y cada una dejó su rastro en History.

#	Caso	Lugar que escribe	Turno	Ruta	Monto
1	Alta hospitalaria, urgente	Huanchaco	Noche	C · válida	190
2	Acompañamiento de mañana	«el centro de Trujillo»	Manana	C · válida	90
3	Día completo post-operatorio	Moche	Dia completo	C · válida	175
4	Distrito sin personal	Laredo	Tarde	B · fuera de cobertura	—
5	Camino infeliz sin lugar ni turno	—	—	A · dato faltante	—

El monto se verifica contra la base: tarifa del turno más recargo de movilidad del distrito. Caso 1: 170 + 20. Caso 2: 90 + 0. Caso 3: 160 + 15. El caso 2 es a propósito una variante del nombre del distrito: el correo dice «el centro de Trujillo» y en Cobertura la fila se llama «Trujillo Centro».

Ruta C · solicitud válida



El recorrido completo: configuración, cobertura filtrada, catálogo, clasificación, RAG, redacción, creación de la solicitud vinculada y pedido de aprobación en Slack. Las rutas A y B quedan sin burbuja.

Ruta B · fuera de cobertura



Laredo está en la tabla Cobertura con Cubierto = No, así que el módulo [4] no lo inyecta en el prompt y la clave distrito queda vacía. El router manda el bundle por la ruta B; no se llama al redactor, se registra Rechazado y se le responde a la familia.

[illegible]

El punto de validación humana en Slack

Escenario 2 · envío tras la aprobación

Evidencia pendiente: evidencias/escenario-2-envio.png.

El panel con datos reales

Evidencia pendiente: evidencias/dashboard-con-datos.png.

Cómo reproducirlo

Los blueprints y el diagrama no están hechos a mano: se generan.

```
# los dos blueprints de Make
python herramientas/build.py

# el mapa de arquitectura (svg + png)
pip install cairosvg
python herramientas/diagrama.py

# este PDF
python herramientas/hacer_pdf_final.py
```

Para levantarlo en otra cuenta de Make: crear un escenario nuevo, menú (...), Import blueprint, subir el json. Hay que volver a apuntar las cuatro conexiones —Airtable, Gmail, Slack y Make AI— a las propias, porque los ids de conexión del blueprint pertenecen a esta cuenta.

Qué aprendí construyéndolo

La base de datos no es el final del flujo, es el medio. Airtable no guarda el resultado: guarda las reglas con las que se toma la decisión. La cobertura y el catálogo de turnos entran al prompt en cada corrida, así que el comportamiento del sistema cambia editando una fila, no un escenario. Eso es lo que separa una automatización de un script con pasos.

A la IA hay que darle la tarea más chica posible. La primera versión le pedía que extrajera datos y evaluara la regla de cobertura y decidiera la ruta. Fallaba. La versión que funciona le pide solo extraer: la base filtra los distritos cubiertos antes de armar el prompt y el router deriva la ruta de los campos extraídos. El modelo pasó de tomar decisiones de negocio a leer un correo, que es lo único que sabe hacer bien.

La pausa humana no es un freno, es el producto. Al principio la pensé como un requisito de la consigna. Terminó siendo la parte que más defiendo: en un negocio donde una familia entrega la llave de su casa, que una persona lea el correo antes de que salga no es fricción, es la razón por la que confían.

Medir cambia la respuesta. Iba a entregar esto con el clasificador en el modelo más barato y una tabla de precios que lo justificaba. El test de estrés lo tiró abajo en tres de cinco corridas. La tabla de costos que quedó vale más que la anterior, no porque el número sea mejor, sino porque el número salió de correr el flujo.

Lo que más me hizo pensar

Los handlers de error no son todos iguales, y esa es la parte interesante. Break cuando reintentar arregla el problema; Resume cuando perder el dato es peor que entregarlo incompleto; ningún reintento cuando el problema está en la entrada y no en el sistema. Decidir cuál va en cada nodo obliga a preguntarse qué es lo peor que puede pasar en ese punto exacto.